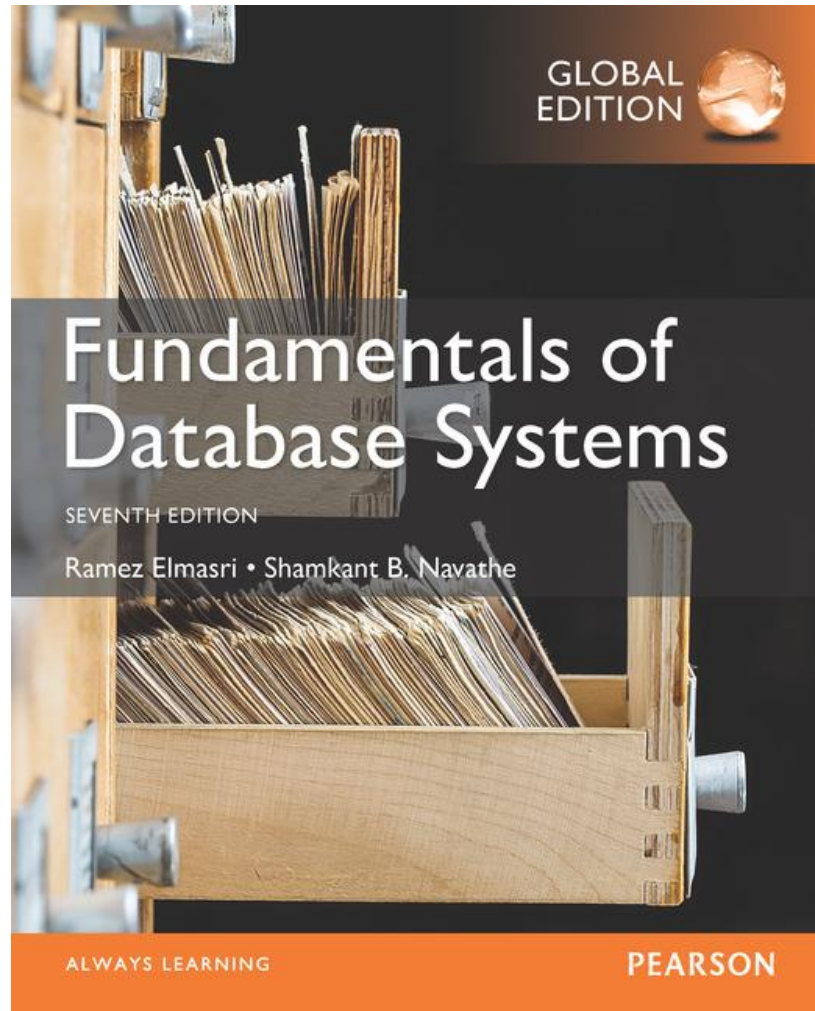




CHAPTER 17

Indexing Structures for Files and Physical Database Design

Introduction

- Indexes used to speed up record retrieval in response to certain search conditions
- Index structures provide secondary access paths
- Any field can be used to create an index
 - Multiple indexes can be constructed
- Most indexes based on ordered files
 - Tree data structures organize the index

17.1 Types of Single-Level Ordered Indexes

- Ordered index similar to index in a textbook
- Indexing field (attribute)
 - Index stores each value of the index field with list of pointers to all disk blocks that contain records with that field value
- Values in index are ordered
- Primary index
 - Specified on the ordering key field of ordered file of records

Types of Single-Level Ordered Indexes (cont'd.)

- Clustering index
 - Used if numerous records can have the same value for the ordering field
- Secondary index
 - Can be specified on any nonordering field
 - Data file can have several secondary indexes

How do you create indexes in ORACLE?

```
CREATE [ UNIQUE ] INDEX <index name>  
ON <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )  
[ CLUSTER ] ;
```

Examples from COMPANY DB

```
CREATE INDEX ssn_index  
ON EMPLOYEE (ssn);
```

```
SELECT ssn  
FROM EMPLOYEE  
WHERE ssn=...
```

```
CREATE index_name  
ON EMPLOYEE (fname,binitial,lname);
```

```
SELECT fname, binitial, lname  
FROM EMPLOYEE  
WHERE fname=... and binitial=...and lname =...
```

```
DROP INDEX ssn_index ON EMPLOYEE;
```

Primary Indexes

- Ordered file with two fields
 - Primary key, $K(i)$
 - Pointer to a disk block, $P(i)$
- One index entry in the index file for each block in the data file
- Indexes may be dense or sparse
 - Dense index has an index entry for every search key value in the data file
 - Sparse index has entries for only some search values (such as indexes on ordering keys)

Primary Indexes (cont'd.)

- ✓ The total number of entries in the Index file = total number of disk blocks in the ordered file.
- ✓ The first record in each block is called the anchor record.

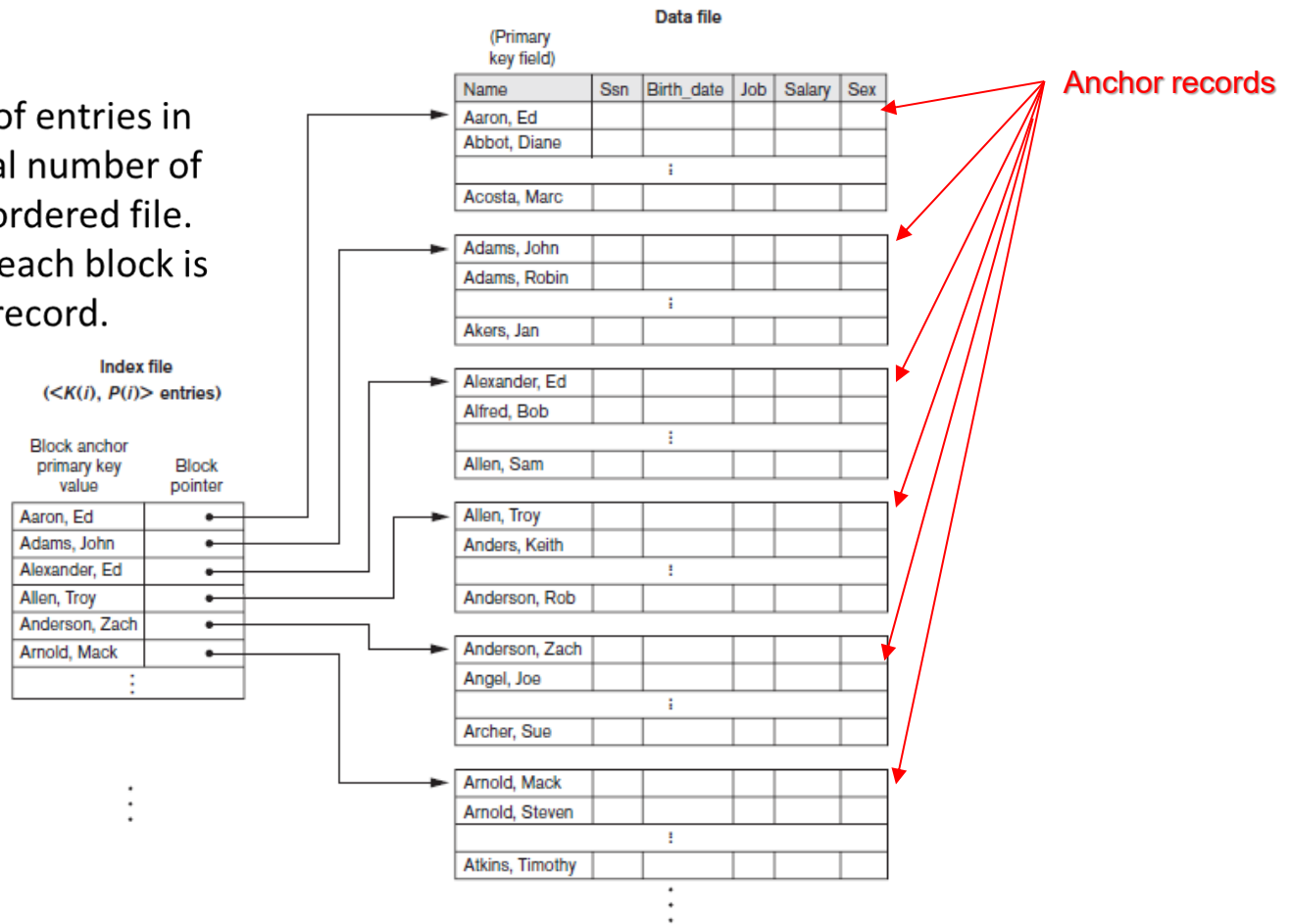


Figure 17.1 Primary index on the ordering key field of the file shown in Figure 16.7

Improving performance of search using Indexing

Example:

Without indexing:

Ordered file with $r=30,000$ records

Disk block size $B=1024$ bytes

Records of fixed size of length $R=100$ bytes

Blocking factor $bfr = B/R = 1024/100 = 10$ records per block (*round down to an int*)

Number of blocks needed for the file $b = r/bfr = 30000/10 = 3000$ blocks (*round up to next int*)

A binary search on the file need approximately $\log_2 b = \log_2 30000 = 12$ block access (*round up to next int*)

Indexing Approach:

Suppose that length of ordering key $V=9$ byte

A block pointer $P=6$ bytes

Size of each entry in index file $R_i = 9+6 = 15$ byte

Blocking factor $bfri = B/R_i = 1024/15 = 68$ entries per block

The total number of index entries ri is equal to the number of blocks $= 3000$

The number of index blocks $bi = ri/bfri = 3000/68 = 45$ blocks

A binary search on index file we need $\log_2 bi = \log_2 45 = 6$ blocks

To search for a record using the index we need one more additional block access

Total $= 6+1 = 7$ block access

Improving performance of search using Indexing

Example 1. Suppose that we have an ordered file with $r = 300,000$ records stored on a disk with block size $B = 4,096$ bytes.⁵ File records are of fixed size and are unspanned, with record length $R = 100$ bytes. The blocking factor for the file would be $bfr = \lfloor (B/R) \rfloor = \lfloor (4,096/100) \rfloor = 40$ records per block. The number of blocks needed for the file is $b = \lceil (r/bfr) \rceil = \lceil (300,000/40) \rceil = 7,500$ blocks. A binary search on the data file would need approximately $\lceil \log_2 b \rceil = \lceil (\log_2 7,500) \rceil = 13$ block accesses.

Now suppose that the ordering key field of the file is $V = 9$ bytes long, a block pointer is $P = 6$ bytes long, and we have constructed a primary index for the file. The size of each index entry is $R_i = (9 + 6) = 15$ bytes, so the blocking factor for the index is $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (4,096/15) \rfloor = 273$ entries per block. The total number of index entries r_i is equal to the number of blocks in the data file, which is 7,500. The number of index blocks is hence $b_i = \lceil (r_i/bfr_i) \rceil = \lceil (7,500/273) \rceil = 28$ blocks. To perform a binary search on the index file would need $\lceil \log_2 b_i \rceil = \lceil (\log_2 28) \rceil = 5$ block accesses. To search for a record using the index, we need one additional block access to the data file for a total of $5 + 1 = 6$ block accesses—an improvement over binary search on the data file, which required 13 disk block accesses. Note that the index with 7,500 entries of 15 bytes each is rather small (112,500 or 112.5 Kbytes) and would typically be kept in main memory thus requiring negligible time to search with binary search. In that case we simply make one block access to retrieve the record.

Primary Indexes (cont'd.)

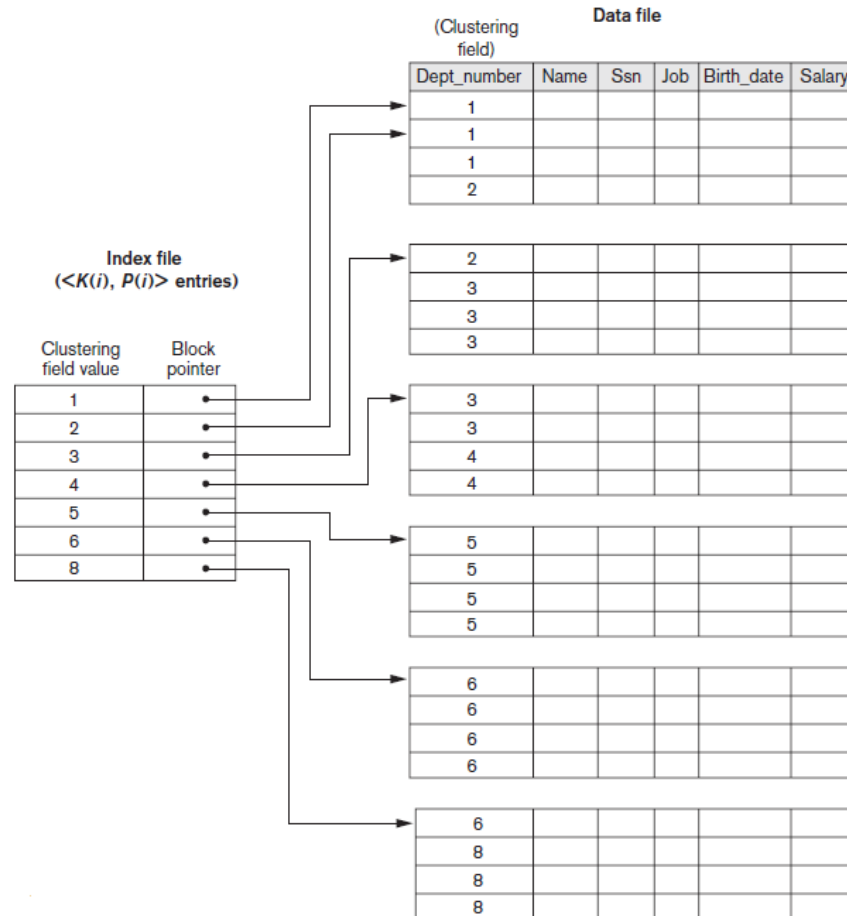
- Major problem: insertion and deletion of records
 - Move records around and change index values
 - Solutions
 - Use unordered overflow file
 - Use linked list of overflow records

Clustering Indexes

- Clustering field
 - File records are physically ordered on a nonkey field without a distinct value for each record
- Ordered file with two fields
 - Same type as clustering field
 - Disk block pointer

Clustering Indexes (cont'd.)

There is one entry in the clustering index for each distinct value of the clustering field, and it contains the value and a pointer to the first block in the data file that has a record with that value for its clustering field.



```
CREATE INDEX DnoIndex  
ON EMPLOYEE (Dno)  
CLUSTER ;
```

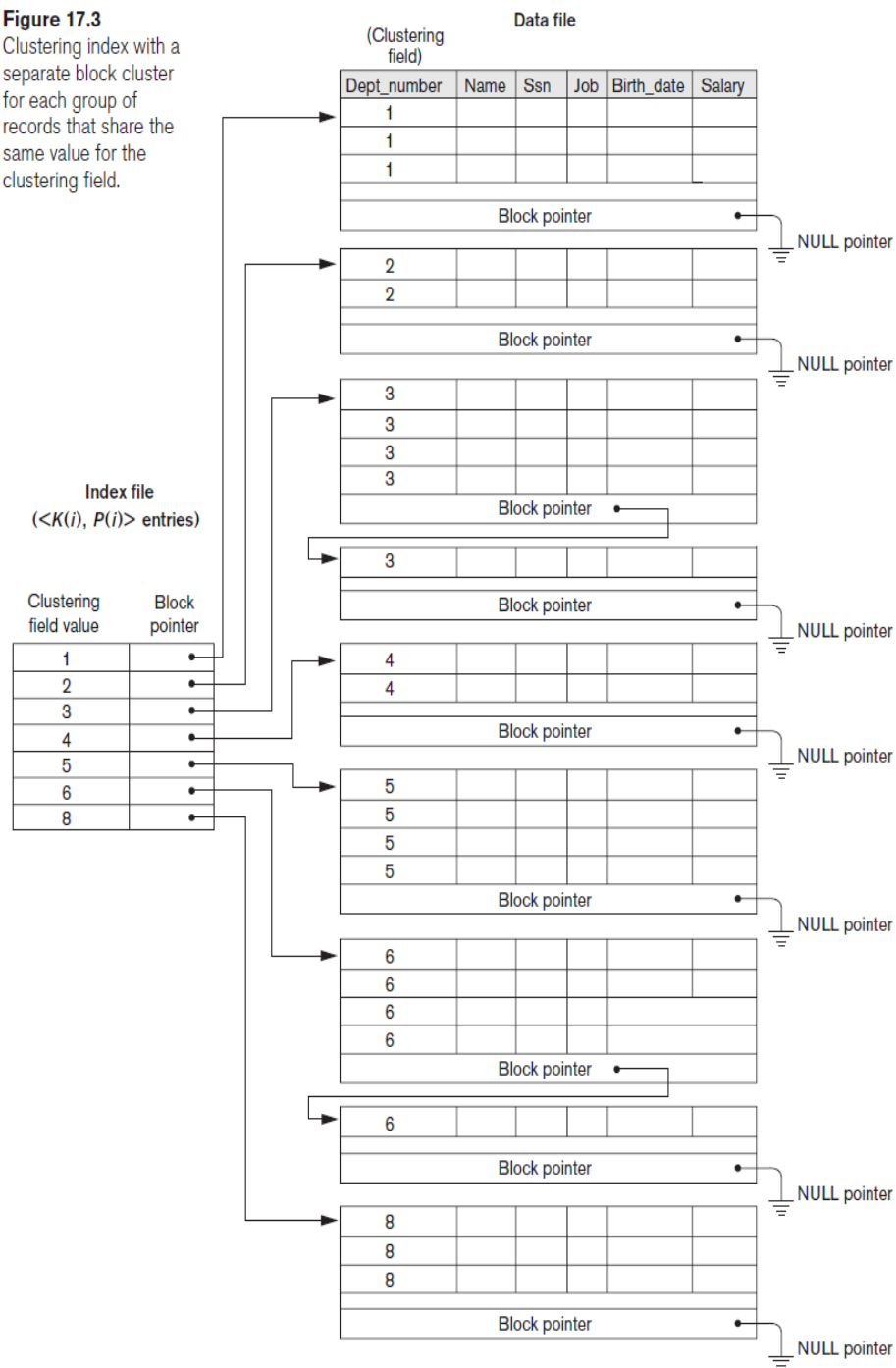
Figure 17.2 A clustering index on the Dept_number ordering nonkey field of an EMPLOYEE file

Clustering Index

Notice that record insertion and deletion still cause problems because the data records are physically ordered. To alleviate the problem of insertion, it is common to reserve a whole block (or a cluster of contiguous blocks) for each value of the clustering field; all records with that value are placed in the block (or block cluster). This makes insertion and deletion relatively straightforward. Figure 17.3 shows this scheme.

Figure 17.3

Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.



Block Accesses using clustering index

Example 2. Suppose that we consider the same ordered file with $r = 300,000$ records stored on a disk with block size $B = 4,096$ bytes. Imagine that it is ordered by the attribute Zipcode and there are 1,000 zip codes in the file (with an average 300 records per zip code, assuming even distribution across zip codes.) The index in this case has 1,000 index entries of 11 bytes each (5-byte Zipcode and 6-byte block pointer) with a blocking factor $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (4,096/11) \rfloor = 372$ index entries per block. The number of index blocks is hence $b_i = \lceil (r_i/bfr_i) \rceil = \lceil (1,000/372) \rceil = 3$ blocks. To perform a binary search on the index file would need $\lceil (\log_2 b_i) \rceil = \lceil (\log_2 3) \rceil = 2$ block accesses. Again, this index would typically be loaded in main memory (occupies 11,000 or 11 Kbytes) and takes negligible time to search in memory. One block access to the data file would lead to the first record with a given zip code.

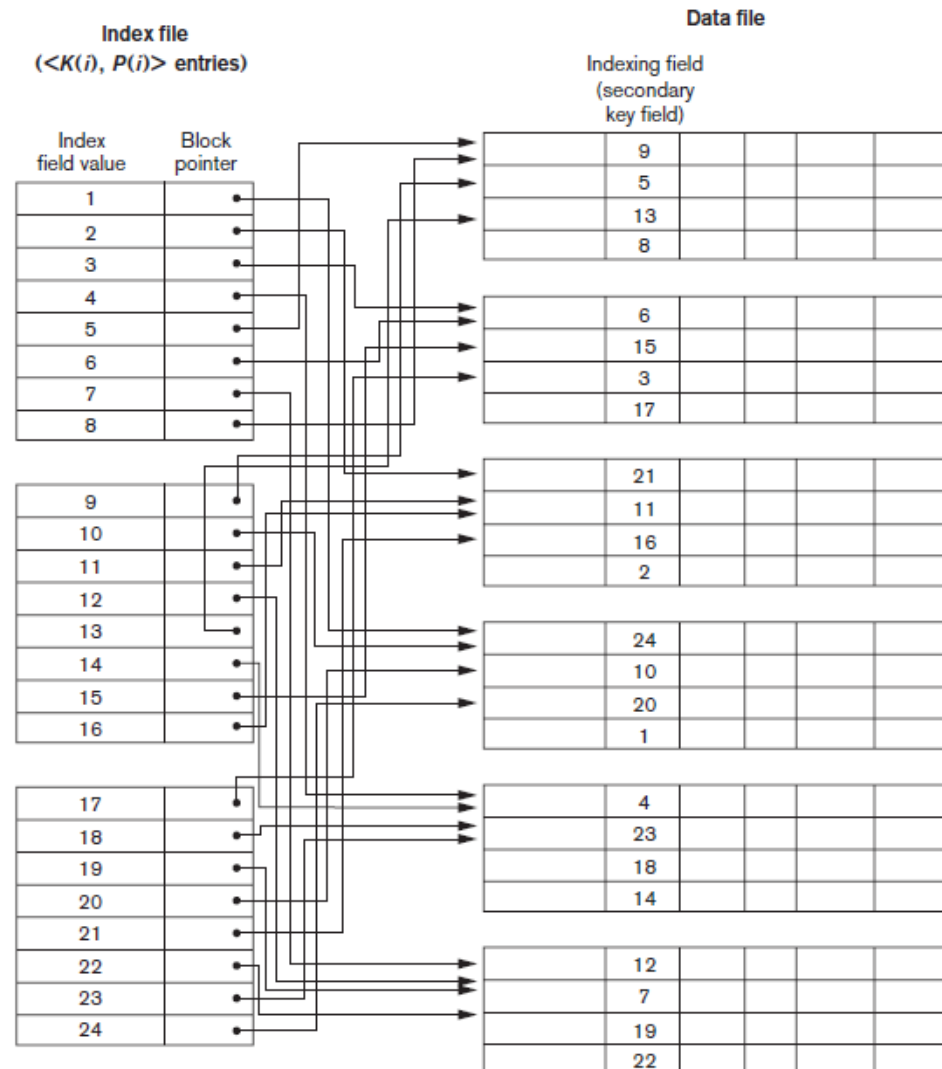
Secondary Indexes

- Provide secondary means of accessing a data file
 - Some primary access exists
- Ordered file with two fields
 - Indexing field, $K(i)$
 - Block pointer or record pointer, $P(i)$
- Usually need more storage space and longer search time than primary index
 - Improved search time for arbitrary record

Notice that a secondary index provides a **logical ordering** on the records by the indexing field. If we access the records in order of the entries in the secondary index, we get them in order of the indexing field. The primary and clustering indexes assume that the field used for **physical ordering** of records in the file is the same as the indexing field.

Secondary Indexes (cont'd.)

Figure 17.4 Dense secondary index (with block pointers) on a nonordering key field of a file.



Block Accesses using secondary index

Example 3. Consider the file of Example 1 with $r = 300,000$ fixed-length records of size $R = 100$ bytes stored on a disk with block size $B = 4,096$ bytes. The file has $b = 7,500$ blocks, as calculated in Example 1. Suppose we want to search for a record with a specific value for the secondary key—a nonordering key field of the file that is $V = 9$ bytes long. Without the secondary index, to do a linear search on the file would require $b/2 = 7,500/2 = 3,750$ block accesses on the average. Suppose that we construct a secondary index on that *nonordering key* field of the file. As in Example 1, a block pointer is $P = 6$ bytes long, so each index entry is $R_i = (9 + 6) = 15$ bytes, and the blocking factor for the index is $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (4,096/15) \rfloor = 273$ index entries per block. In a dense secondary index such as this, the total number of index entries r_i is equal to the *number of records* in the data file, which is 300,000. The number of blocks needed for the index is hence $b_i = \lceil (r_i/bfr_i) \rceil = \lceil (300,000/273) \rceil = 1,099$ blocks.

A binary search on this secondary index needs $\lceil (\log_2 b_i) \rceil = \lceil (\log_2 1,099) \rceil = 11$ block accesses. To search for a record using the index, we need an additional block access to the data file for a total of $11 + 1 = 12$ block accesses—a vast improvement over the 3,750 block accesses needed on the average for a linear search, but slightly worse than the 6 block accesses required for the primary index. This difference arose because the primary index was nondense and hence shorter, with only 28 blocks in length as opposed to the 1,099 blocks dense index here.

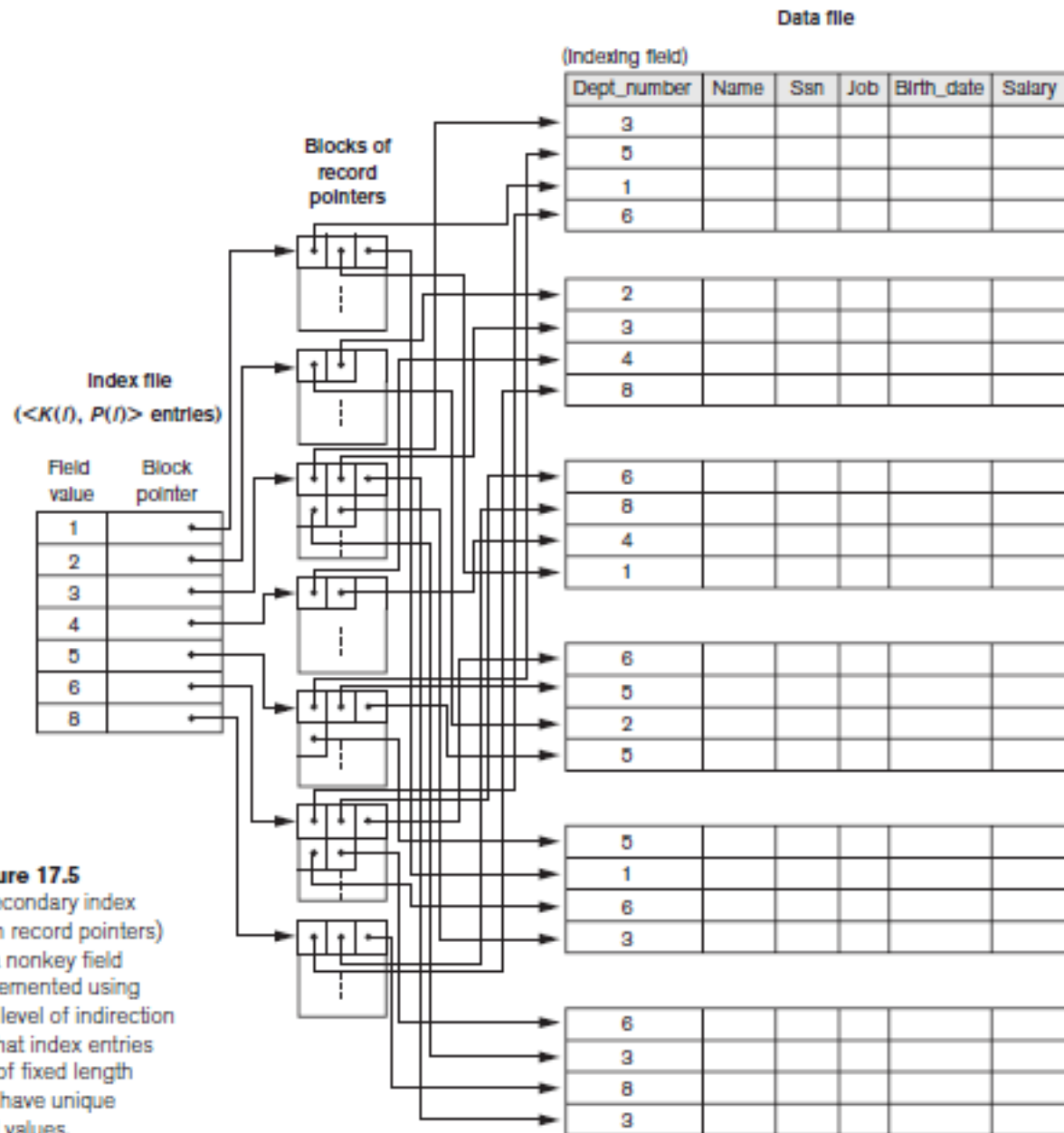


Figure 17.5

A secondary index (with record pointers) on a nonkey field implemented using one level of indirection so that index entries are of fixed length and have unique field values.

Types of Single-Level Ordered Indexes (cont'd.)

Table 17.1 Types of indexes based on the properties of the indexing field

	Index Field Used for Physical Ordering of the File	Index Field Not Used for Physical Ordering of the File
Indexing field is key	Primary index	Secondary index (Key)
Indexing field is nonkey	Clustering index	Secondary index (NonKey)

Type of Index	Number of (First-Level) Index Entries	Dense or Nondense (Sparse)	Block Anchoring on the Data File
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no ^a
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records ^b or number of distinct index field values ^c	Dense or Nondense	No

^aYes if every distinct value of the ordering field starts a new block; no otherwise.

^bFor option 1.

^cFor options 2 and 3.

- Option 1 is to include duplicate index entries with the same $K(i)$ value—one for each record. This would be a dense index.
- Option 2 is to have variable-length records for the index entries, with a repeating field for the pointer. We keep a list of pointers $\langle P(i, 1), \dots, P(i, k) \rangle$ in the index entry for $K(i)$ —one pointer to each block that contains a record whose indexing field value equals $K(i)$. In either option 1 or option 2, the binary search algorithm on the index must be modified appropriately to account for a variable number of index entries per index key value.

Index Creation

- General form of the command to create an index

```
CREATE [ UNIQUE ] INDEX <index name>  
ON <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )  
[ CLUSTER ] ;
```

- Unique and cluster keywords optional
 - Order can be ASC or DESC
- Secondary indexes can be created for any primary record organization
 - Complements other primary access methods

Tuning Indexes

- Tuning goals
 - Dynamically evaluate requirements
 - Reorganize indexes to yield best performance
- Reasons for revising initial index choice
 - Certain queries may take too long to run due to lack of an index
 - Certain indexes may not get utilized
 - Certain indexes may undergo too much updating if based on an attribute that undergoes frequent changes

17.7 Physical Database Design in Relational Databases

- Physical design goals
 - Create appropriate structure for data in storage
 - Guarantee good performance
- Must know job mix for particular set of database system applications
- Analyzing the database queries and transactions
 - Information about each retrieval query
 - Information about each update transaction

Physical Database Design in Relational Databases (cont'd.)

- Analyzing the expected frequency of invocation of queries and transactions
 - Expected frequency of using each attribute as a selection or join attribute
 - 80-20 rule: 80 percent of processing accounted for by only 20 percent of queries and transactions
- Analyzing the time constraints of queries and transactions
 - Selection attributes associated with time constraints are candidates for primary access structures

Physical Database Design in Relational Databases (cont'd.)

- Analyzing the expected frequency of update operations
 - Minimize number of access paths for a frequently-updated file
 - Updating the access paths themselves slows down update operations
- Analyzing the uniqueness constraints on attributes
 - Access paths should be specified on all *candidate key* attributes that are either the primary key of a file or unique attributes

Physical Database Design Decisions

- Design decisions about indexing
 - Whether to index an attribute
 - Attribute is a key or used by a query
 - What attribute(s) to index on
 - Single or multiple
 - Whether to set up a clustered index
 - One per table